

Python on the IBMi- Chapter 9



At the edge of a quiet clearing, our hero meets a figure cloaked in legacy, the old RPG programmer. Weathered by decades of syntax and subfiles, he offers no warning, only a knowing nod and the torch. It's time. The next step in this journey isn't about fixing what was, but building what's next.

In this chapter, we'll add the ability to create new records in our Product Master. From constructing the front-end form to validating input and writing the data using RPG, we'll empower our application to grow. This is more than just another feature, it's the beginning of a new era, where the tools of the past meet the creativity of the future.

Disclaimer

This article will use the built-in Python HTTP Server. While this works, it is NOT production ready. While it might work for simple utilities and very low demand. It's a very simple HTTP Server that handles basic requests. The next few articles are for demonstration purposes only.

Prerequisites

- Please refer to chapter 1 for other prerequisites (<https://www.linkedin.com/pulse/python-ibmi-chapter-1-ricky-thompson-zg7te/>)
- Code can be found here: (https://github.com/thomprl/ibmi_python_rpg_adventures)

Objective

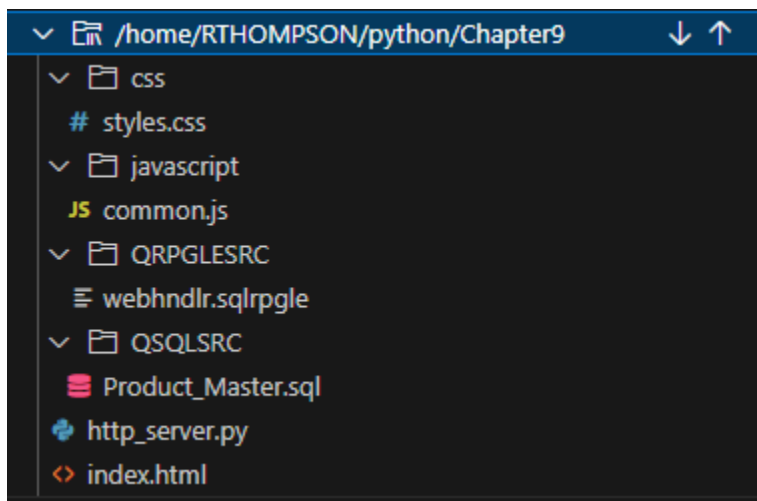
- Add the ability to add a new product master record

Before We Begin

Again there will be a good amount of code used in this article. Most of it is repeated with updates from the previous chapters but it will be hard to explain in detail what each line of code is doing.

- No changes to the CSS file (style.css).
- No changes to the Python code (http_server.py).
- The JavaScript file (common.js) we add just some code to open the Product Modal on clicking the add button.
- One minor change to the HTML file (index.html) to add a button to Add a New Product.
- The RPG program (webhndlr.sqlrpgle) we add the code to handle writing a new record and reuse our edit procedure as well.

This might come in handy as well. Each article I make a new IFS folder and put all the contents into it. This is a picture of my (/home/rthompson/python/chapter9) folder from VS Code.



Let's Code

I'm only going to show the programs that actually had changes done to cut down on the length. If you need to find the previous articles you can get the source and a PDF version off of GitHub.

common.js

Added a new function called `addNewProduct` that will Open the Product Modal to the end of the (**common.js**) file

```
let notificationTimeout;

function showNotification(message, type) {
    const notification =
document.getElementById('notification');
    const messageElement =
document.getElementById('notification-message');
    const modal = document.getElementById('productModal');

    // Clear any existing timeout
    if (notificationTimeout) {
        clearTimeout(notificationTimeout);
    }

    // Remove previous type classes
    notification.classList.remove('success', 'error');

    // Add new type class
    notification.classList.add(type);

    // Position notification based on modal visibility
    if (modal.style.display === 'block') {
        notification.classList.add('over-modal');
    } else {
        notification.classList.remove('over-modal');
    }

    messageElement.textContent = message;
    notification.style.display = 'block';

    // Auto-hide after 10 seconds
    notificationTimeout = setTimeout(hideNotification, 10000);
}

function hideNotification() {
    const notification =
document.getElementById('notification');
    notification.style.display = 'none';
    notification.classList.remove('over-modal');

    // Clear the timeout when manually closing
    if (notificationTimeout) {
```

```

        clearTimeout(notificationTimeout);
        notificationTimeout = null;
    }
}

function deleteProduct(productNumber) {
    if (confirm(`Are you sure you want to delete product:
${productNumber}?`)) {
        $.ajax({
            url: '/api',
            type: 'POST',
            contentType: 'application/json',
            data: JSON.stringify({
                app: 'product',
                action: 'delete',
                productnumber: productNumber
            }),
            success: function (response) {
                if (response.success) {
                    $('#myTable').DataTable().ajax.reload();
                    showNotification(response.message,
'success');
                } else {
                    showNotification(response.message, 'error');
                }
            },
            error: function (xhr, status, error) {
                showNotification('Error deleting product: ' +
error, 'error');
            }
        });
    }
}

// Modal logic
function openProductModal(productNumber) {
    // Clear all form fields first
    $('#modalProductNumber').val('');
    $('#modalDescription').val('');
    $('#modalCost').val('');
    $('#modalUOM').val('');
    $('#modalCategory').val('');

    document.getElementById('productModal').style.display =
'block';
    if (productNumber) {
        // Edit mode
    }
}

```

```

        $('#modalTitle').text('Edit Product');
        $('#modalProductNumber').prop('readonly', true);
        $.ajax({
            url:
`/api?app=product&action=getproduct&productnumber=${productNumber}`
            ,
            type: 'GET',
            success: function (response) {
                // response.data contains the product record
                const product = response.data;

                $('#modalProductNumber').val(product.ProductNumber);
                $('#modalDescription').val(product.Description);
                $('#modalCost').val(product.Cost);
                $('#modalUOM').val(product.UnitOfMeasure);
                $('#modalCategory').val(product.Category);
            },
            error: function () {
                showNotification('Error retrieving product.',
'error');
                closeProductModal();
            }
        });
    } else {
        // Add mode
        $('#modalTitle').text('Add Product');
        $('#modalProductNumber').prop('readonly',
false).val('');
        $('#modalDescription').val('');
        $('#modalCost').val('');
        $('#modalUOM').val('');
        $('#modalCategory').val('');
    }
}

function closeProductModal() {
    document.getElementById('productModal').style.display =
'none';
}

function submitProductForm() {
    // Force cost to 0 if blank
    if ($('#modalCost').val() === '') {
        $('#modalCost').val(0);
    }
    let action = ($('#modalTitle').text().startsWith('Edit') ?
'updateproduct' : 'addproduct';

```

```

const productData = {
    app: 'product',
    action: action,
    ProductNumber: $('#modalProductNumber').val(),
    Description: $('#modalDescription').val(),
    Cost: $('#modalCost').val(),
    UnitOfMeasure: $('#modalUOM').val(),
    Category: $('#modalCategory').val()
};

$.ajax({
    url: '/api',
    type: 'POST',
    contentType: 'application/json',
    data: JSON.stringify(productData),
    success: function (response) {
        if (response.success) {
            $('#myTable').DataTable().ajax.reload();
            showNotification(response.message, 'success');
            closeProductModal();
        } else {
            showNotification(response.message, 'error');
        }
    },
    error: function (xhr, status, error) {
        showNotification('Error saving product: ' + error,
'error');
    }
});
}

$(document).ready(function () {
    var table = $('#myTable').DataTable({
        ajax: {
            url: '/api?app=product&action=getproductlist',
            type: 'GET',
            dataSrc: 'data'
        },
        columns: [
            { 'data': 'ProductNumber' },
            { 'data': 'Description' },
            { 'data': 'Cost' },
            { 'data': 'UnitOfMeasure' },
            { 'data': 'Category' },
            {
                'data': null,
                'render': function (data, type, row) {

```

```

        return `
            <div class="action-buttons">
                <i class="fas fa-edit"
title="Edit"
onclick="openProductModal('${row.ProductNumber}')"></i>
                <i class="fas fa-trash-alt"
onclick="deleteProduct('${row.ProductNumber}')"
title="Delete"></i>
            </div>
        `;
    }
}
],
pageLength: 10,
order: [[1, 'asc']]
});
});

// Add this function near the other modal functions
function addNewProduct() {
    openProductModal(); // Call existing openProductModal with
no parameters
}

```

index.html

Added a new option on the menu bar to Add New Product.

```

<!DOCTYPE html>
<html>

<head>
    <meta charset="UTF-8">
    <title>Product List</title>

    <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.m
in.js"></script>
    <link href="https://cdn.datatables.net/v/dt/dt-
2.3.2/datatables.min.css" rel="stylesheet"
        integrity="sha384-
d76uxpdVr9QyCSR9vVSYd0AZeRzNUN8A4JVqUHBVXyGxZ+oOfrZVHC/1Y58mhyNg
" crossorigin="anonymous">
    <script src="https://cdn.datatables.net/v/dt/dt-
2.3.2/datatables.min.js"

```

```
        integrity="sha384-
JRUjeYWWUG0171YFugrU0ksSC6CaWnl4XzwP6mNjnnDh4hfFGRyYbEXwryGwLsEp
"
        crossorigin="anonymous"></script>
        <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.5.1/css/all.min.css">

        <link rel="stylesheet" href="css/styles.css">
        <script src="javascript/common.js"></script>
</head>

<body>
    <!-- Product Modal -->
    <div id="productModal" class="modal">
        <div class="modal-content">
            <span class="close"
onclick="closeProductModal()">&times;</span>
            <h2 id="modalTitle">Edit Product</h2>
            <form id="productForm">
                <label for="modalProductNumber">Product
Number:</label>
                <input type="text" id="modalProductNumber"
name="ProductNumber" readonly
                style="background:#eee;"><br><br>
                <label
for="modalDescription">Description:</label>
                <input type="text" id="modalDescription"
name="Description"><br><br>
                <label for="modalCost">Cost:</label>
                <input type="number" id="modalCost" name="Cost"
min=0 required value="0" step="0.01"><br><br>
                <label for="modalUOM">UOM:</label>
                <input type="text" id="modalUOM"
name="UnitOfMeasure"><br><br>
                <label for="modalCategory">Category:</label>
                <input type="text" id="modalCategory"
name="Category"><br><br>
                <button type="button"
onclick="submitProductForm()">Submit</button>
                <button type="button"
onclick="closeProductModal()">Cancel</button>
            </form>
        </div>
    </div>

    <!-- Navigation Bar -->
```



```

<div class="navbar">
  <a href="/">Home</a>
  <div class="dropdown">
    <button class="dropbtn">Tools
      <i class="fa fa-caret-down"></i>
    </button>
    <div class="dropdown-content">
      <a
href="/api?app=product&action=getproductlist">Get Raw JSON
Product List</a>
      <a href="/api?action=shutdown">Shutdown Web
Server</a>
    </div>
  </div>
  <a href="#" onclick="addNewProduct()">Add New
Product</a>
</div>

<br><br>

<div id="notification" style="display:none;">

  <span id="notification-message"></span>
  <button onclick="hideNotification()"
style="float:right;">&times;</button>
</div>
<br>
<div>
  <table id="myTable" class="display table table-striped
table-bordered" style="width:100%">
    <thead>
      <tr>
        <th>Product Number</th>
        <th>Description</th>
        <th>Cost</th>
        <th>UOM</th>
        <th>Category</th>
        <th>Actions</th>
      </tr>
    </thead>
  </table>
</div>
</body>

</html>

```

webhndlr.sqlrpgle

In the RPG we check to see what the action type is and call the AddProduct subprocedure. This will return any error messages and whether it's successful or not. If you try to add the same product number it will display an error.

```
**free
Ctl-Opt option(*nodebugio:*srcstmt:*nounref) Main(Main)
dftactgrp(*no)
  text('Web Server RPG Handler Program');

// The main procedure that will listen for incoming requests
Dcl-Proc Main;
  Dcl-Pi *N;
    pMethod CHAR(10) const;
    pRequestData CHAR(32000) const;
    pResponseData CHAR(32000);
  End-Pi;

  Dcl-S vJsonString CHAR(32000);
  Dcl-S vApp VARCHAR(50);
  Dcl-S vAction VARCHAR(50);
  // Dcl-S vRequestData SQLTYPE(CLOB:500000);

  Exec Sql
    Set Option commit = *none;

    // If QueryString is blank display main page from the IFS
  If pRequestData = '';
    pResponseData = '{"data": null}';
    Return;
  Else;
    Exec Sql
      SELECT upper(jt.app)
             ,upper(jt.action)
      INTO :vApp
           ,:vAction
      FROM JSON_TABLE(trim(:pRequestData),
                     'lax $'
                     COLUMNS (
                           app      VARCHAR(50) PATH 'lax $.app',
                           action  VARCHAR(50) PATH 'lax $.action'
                         )
           ) AS jt;

    If sqlcod <> 0;
```

```

        pResponseData = '{ "success": false, "message": "Invalid
JSON format" }';
        Return;
    EndIf;
EndIf;

If %upper(pMethod) = 'GET ';
    Select;
        When vApp = 'PRODUCT' and vAction = 'GETPRODUCTLIST';
            // Call the procedure to get the Product List in JSON
format
            vJsonString = GetProductList();
            pResponseData = vJsonString;
        When vApp = 'PRODUCT' and vAction = 'GETPRODUCT';
            // Call the procedure to get the Product List in JSON
format
            vJsonString = GetProduct(pRequestData);
            pResponseData = vJsonString;
        Other;
            pResponseData = '{"success": false, "message": "Unknown
Action Sent"}';
        EndSl;
    EndIf;

If %upper(pMethod) = 'POST';
    Select;
        When vApp = 'PRODUCT' and vAction = 'DELETE';
            vJsonString = DeleteProduct(pRequestData);
            pResponseData = vJsonString;
        When vApp = 'PRODUCT' And (vAction = 'UPDATEPRODUCT');
            vJsonString = UpdateProduct(pRequestData : vAction);
            pResponseData = vJsonString;
        When vApp = 'PRODUCT' And (vAction = 'ADDPRODUCT');
            vJsonString = AddProduct(pRequestData : vAction);
            pResponseData = vJsonString;
        Other;
            pResponseData = '{"success": false, "message":' +
                pMethod + '/' + vAction + '" Unknown Action Sent"}';
        EndSl;
    EndIf;

End-Proc;

Dcl-Proc GetProductList;
    Dcl-Pi *N CHAR(32000);
    End-Pi;

```

```

Dcl-S jsonString CHAR(32000);
Dcl-S jsonStringClob SQLTYPE(CLOB:32000);

Exec Sql
Select Cast(
    Json_Object(
        'data' Value (Json_Arrayagg(
            Json_Object(
                'ProductNumber' Value rtrim(product_number)
                , 'Description' Value Rtrim(description)
                , 'Cost' Value Cost
                , 'UnitOfMeasure' Value Rtrim(unit_of_measure)
                , 'Category' Value Rtrim(category)
            )
        ))
    ) As Clob(32000))
Into :jsonstringclob
From rthompson1.product_master;

JsonString = JsonStringClob_Data;
Return JsonString;
End-Proc;

Dcl-Proc GetProduct;
Dcl-Pi *N CHAR(32000);
    pRequestData CHAR(32000) const;
End-Pi;

Dcl-S jsonString CHAR(32000);
Dcl-S jsonStringClob SQLTYPE(CLOB:32000);
Dcl-S vProductNumber VARCHAR(20);

Exec Sql
    SELECT jt.productnumber
    INTO :vProductNumber
    FROM JSON_TABLE(trim(:pRequestData),
        'lax $'
        COLUMNS (
            productnumber VARCHAR(20) PATH 'lax $.productnumber'
        )
    ) AS jt;

If sqlcod <> 0;
    jsonString = '{"success": false, "message": "Invalid JSON
format"}';
    Return jsonString;
EndIf;

```

```

Exec Sql
  Select Cast(
    Json_Object(
      'data' Value
        Json_Object(
          'ProductNumber' Value rtrim(product_number)
          , 'Description' Value Rtrim(description)
          , 'Cost' Value Cost
          , 'UnitOfMeasure' Value Rtrim(unit_of_measure)
          , 'Category' Value Rtrim(category)
        )
      )
    As CLOB(32000))
  Into :jsonstringclob
  From rthompson1.product_master
  Where product_number = :vProductNumber
  Fetch First Row Only;

If sqlcod = 100;
  jsonString = '{"success": false, "message": "Product not
found"}';
  Return jsonString;
EndIf;

If sqlcod <> 0;
  jsonString = '{"success": false, "message": "Error Occurred
while retrieving product"}';
  Return jsonString;
EndIf;

JsonString = JsonStringClob_Data;
Return JsonString;
End-Proc;

Dcl-Proc DeleteProduct;
  Dcl-Pi *N CHAR(32000);
    pRequestData CHAR(32000) const;
  End-Pi;

  Dcl-S vProductNumber VARCHAR(20);
  Dcl-S vJsonResponse VARCHAR(32000);

  Exec Sql
    SELECT jt.productnumber
    INTO :vProductNumber
    FROM JSON_TABLE(trim(:pRequestData),

```

```

        'lax $'
        COLUMNS (
            productnumber VARCHAR(20) PATH 'lax $.productnumber'
        )
    ) AS jt;

If sqlcod <> 0;
    vJsonResponse = '{"success": false, "message": "Invalid JSON
format"}' ;
    Return vJsonResponse;
EndIf;

// Perform the delete operation
Exec Sql
    DELETE FROM rthompson1.product_master
    WHERE product_number = :vProductNumber;

If sqlcod = 0;
    vJsonResponse = '{"success": true, "message": "Successfully
deleted product: '
        + %trim(vProductNumber) + '" }';
Else;
    vJsonResponse = '{"success": false, "message": "Failed to
delete product: '
        + %trim(vProductNumber) + '" }';
EndIf;

Return vJsonResponse;
End-Proc;

Dcl-Proc UpdateProduct;
    Dcl-Pi *N CHAR(32000);
        pRequestData CHAR(32000) const;
        pAction VARCHAR(50) const;
    End-Pi;

    Dcl-S vJsonResponse   VARCHAR(32000);
    Dcl-S vProductNumber  VARCHAR(20);
    Dcl-S vDescription     VARCHAR(100);
    Dcl-S vCost            PACKED(10:2);
    Dcl-S vUnitOfMeasure  VARCHAR(10);
    Dcl-S vCategory        VARCHAR(50);
    Dcl-S vExists          IND INZ(*Off);

    Exec Sql
        SELECT jt.productnumber
            ,jt.description

```

```

        ,jt.cost
        ,jt.unitofmeasure
        ,jt.category
    INTO :vProductNumber
        ,:vDescription
        ,:vCost
        ,:vUnitOfMeasure
        ,:vCategory
    FROM JSON_TABLE(trim(:pRequestData),
        'lax $'
        COLUMNS (
            productnumber VARCHAR(20)    PATH 'lax
$.ProductNumber',
            descriptionon  VARCHAR(100)  PATH 'lax
$.Description',
            cost            DECIMAL(10,2) PATH 'lax $.Cost',
            unitofmeasure  VARCHAR(10)    PATH 'lax
$.UnitOfMeasure',
            category        VARCHAR(10)   PATH 'lax $.Category'
        )
    ) AS jt;

    If sqlcod <> 0;
        vJsonResponse = '{"success": false, "message": "Invalid JSON
format"}' ;
        Return vJsonResponse;
    EndIf;

    // Check to see if it exists
    Exec Sql
        Select 1
            Into :vExists
        From rthompson1.product_master
        Where product_number = :vProductNumber
        Fetch First Row Only;

    If sqlcod = 100 or Not vExists;
        vJsonResponse = '{"success": false, "message": "Product not
found"}';
        Return vJsonResponse;
    EndIf;

    // Edit the product records
    If EditProduct(vProductNumber
        :vDescription
        :vCost
        :vUnitOfMeasure

```

```

        :vCategory
        :vJsonResponse
    ) = *On;

    Return vJsonResponse;
EndIf;

Exec Sql
    UPDATE rthompson1.product_master
    SET description = :vDescription
        ,cost = :vCost
        ,unit_of_measure = :vUnitOfMeasure
        ,category = :vCategory
    WHERE product_number = :vProductNumber;

If sqlcod = 0;
    vJsonResponse = '{"success": true, "message":
"Successfully updated product: '
        + %trim(vProductNumber) + '" }';
Else;
    vJsonResponse = '{"success": false, "message": "Failed to
update product: '
        + %trim(vProductNumber) + '" }';
EndIf;

Return vJsonResponse;
End-Proc;

Dcl-Proc EditProduct;
    Dcl-Pi *N IND; // Returns *On if there is an error
        pProductNumber VARCHAR(20) const;
        pDescription VARCHAR(100) const;
        pCost PACKED(10:2) const;
        pUnitOfMeasure VARCHAR(10) const;
        pCategory VARCHAR(50) const;
        pJsonResponse VARCHAR(32000);
    End-Pi;

    Dcl-S vErrorMessage VARCHAR(2000);

    If pDescription = '';
        vErrorMessage = 'Description cannot be blank ' + '\n';
    EndIf;

    If pCost < 0;
        vErrorMessage += 'Cost cannot be negative' + '\n';
    EndIf;

```



```

If pUnitOfMeasure = '';
    vErrorMessage += 'Unit of Measure cannot be blank' + '\n';
EndIf;

If pCategory = '';
    vErrorMessage += 'Category cannot be blank' + '\n';
EndIf;

If vErrorMessage <> '';
    pJsonResponse = '{"success": false, "message": "' +
vErrorMessage + '"}';
    Return *On;
EndIf;
Return *Off;
End-Proc;

Dcl-Proc AddProduct;
    Dcl-Pi *N CHAR(32000);
        pRequestData CHAR(32000) const;
        pAction VARCHAR(50) const;
    End-Pi;

    Dcl-S vJsonResponse    VARCHAR(32000);
    Dcl-S vProductNumber   VARCHAR(20);
    Dcl-S vDescription      VARCHAR(100);
    Dcl-S vCost             PACKED(10:2);
    Dcl-S vUnitOfMeasure   VARCHAR(10);
    Dcl-S vCategory         VARCHAR(50);
    Dcl-S vExists          IND INZ(*Off);

    Exec Sql
        SELECT jt.productnumber
            ,jt.descriptieon
            ,jt.cost
            ,jt.unitofmeasure
            ,jt.category
        INTO :vProductNumber
            ,:vDescription
            ,:vCost
            ,:vUnitOfMeasure
            ,:vCategory
        FROM JSON_TABLE(trim(:pRequestData),
            'lax $'
            COLUMNS (
                productnumber VARCHAR(20)    PATH 'lax
$.ProductNumber',

```

```

        description    VARCHAR(100)    PATH 'lax
$.Description',
        cost           DECIMAL(10,2)    PATH 'lax $.Cost',
        unitofmeasure  VARCHAR(10)      PATH 'lax
$.UnitOfMeasure',
        category       VARCHAR(10)      PATH 'lax $.Category'
    )
    ) AS jt;

If sqlcod <> 0;
    vJsonResponse = '{"success": false, "message": "Invalid JSON
format"}' ;
    Return vJsonResponse;
EndIf;

// Check to see if it exists
Exec Sql
    Select 1
        Into :vExists
    From rthompson1.product_master
    Where product_number = :vProductNumber
    Fetch First Row Only;

If sqlcod = 0 or vExists;
    vJsonResponse = '{"success": false, "message": "Product
Number already exists"}';
    Return vJsonResponse;
EndIf;

// Edit the product records
If EditProduct(vProductNumber
                :vDescription
                :vCost
                :vUnitOfMeasure
                :vCategory
                :vJsonResponse
                ) = *On;

    Return vJsonResponse;
EndIf;

Exec Sql
    INSERT INTO rthompson1.product_master
        (product_number
        ,description
        ,cost
        ,unit_of_measure

```

```

        ,category)
VALUES
    (:vProductNumber
    ,:vDescription
    ,:vCost
    ,:vUnitOfMeasure
    ,:vCategory);

If sqlcod = 0;
    vJsonResponse = '{"success": true, "message":
"Successfully added product: '
                    + %trim(vProductNumber) + '" }';
Else;
    vJsonResponse = '{"success": false, "message": "Failed to
add product: '
                    + %trim(vProductNumber) + '" }';
EndIf;

Return vJsonResponse;

End-Proc;

```

Now we have all the code and we have compiled the SQLRPGLE program (hopefully successfully) we are ready to give this a go.

Start the Web Server

Make sure the port you're using (e.g., 43001) is available on your IBMi.

From VS Code Open a PASE terminal by pressing (Ctrl+Shift+J) and selecting PASE and then type the following:

```

cd python/chapter8
python3 http_server.py --port 43001 --library RTHOMPSON1

```

It will take a few seconds to start the server. You will see a message in the terminal similar to this.

```

-bash-5.2$ python3 http_server.py --port 43001 --library RTHOMPSON1
Starting HTTP Server... Please wait...
Serving on http://localhost:43001 using library RTHOMPSON1

```

Now open a web browser and go to your IBMi name or IP address plus the port number.

```

http://myibmi:43001/

```

You should see web page open with a list of products being returned from our RPG program. Notice the new Add New Product button at the top.

Home	Tools ▾	Add New Product
------	---------	-----------------

10 ▾ entries per page

Search:

Product Number	Description	Cost	UOM	Category	Actions
P0005	16GB USB Drive	9.99	EACH	Electronics	 
P0013	AA Batteries (4 Pack)	4.25	PACK	Electronics	 
P0008	Ballpoint Pens (Pack of 10)	2.99	PACK	Stationery	 
P0002	Bluetooth Keyboard	29.95	EACH	Electronics	 
P0006	Coffee Mug	5.25	EACH	Kitchen	 
P0009	Desk Lamp	18.75	EACH	Office	 
P0020	Electric Kettle	24.5	EACH	Kitchen	 
P0004	Laptop Backpack	45	EACH	Accessories	 
P0016	LED Light Bulbs (6 Pack)	12.49	PACK	Home Improvement	 
P0019	Mechanical Pencil Set (2 Pack)	4.75	PACK	Stationery	 

Showing 1 to 10 of 20 entries

« < 1 2 > »

Once we click the Add New Product button a modal will open up allowing us to enter information.

[Home](#) [Tools](#) [Add New Product](#)

10 entries per page

Search:

Product Number	Description	Category	Actions
P0005	16GB USB	Electronics	Edit Delete
P0013	AA Batteries	Electronics	Edit Delete
P0008	Ballpoint Pen	Stationery	Edit Delete
P0002	Bluetooth Headphones	Electronics	Edit Delete
P0006	Coffee Maker	Kitchen Appliances	Edit Delete
P0009	Desk Lamp	Home Improvement	Edit Delete
P0020	Electric Drill	Tools	Edit Delete
P0004	Laptop Bag	Accessories	Edit Delete
P0016	LED Light Bulbs	Home Improvement	Edit Delete
P0019	Mechanical Pencil	Stationery	Edit Delete

Showing 1 to 10 of 20 entries

×

Add Product

Product Number:

Description:

Cost:

UOM:

Category:

If I key in an existing product number and click submit you will see an error message appear.

[Home](#) [Tools](#) [Add New Product](#)

10 entries per page

Search:

Product Number	Description	Category	Actions
P0005	16GB USB	Electronics	Edit Delete
P0013	AA Batteries	Electronics	Edit Delete
P0008	Ballpoint Pen	Stationery	Edit Delete
P0002	Bluetooth Headphones	Electronics	Edit Delete
P0006	Coffee Maker	Kitchen Appliances	Edit Delete
P0009	Desk Lamp	Home Improvement	Edit Delete
P0020	Electric Drill	Tools	Edit Delete
P0004	Laptop Bag	Accessories	Edit Delete
P0016	LED Light Bulbs	Home Improvement	Edit Delete
P0019	Mechanical Pencil	Stationery	Edit Delete

Showing 1 to 10 of 20 entries

×

Add Product

Product Number:

Description:

Cost:

UOM:

Category:

Product Number already exists

If we put in a good product number that doesn't already exist but leave the Description, UOM, Category blank and click submit the error notification will display several messages.

The screenshot shows a web application interface with a dark header bar containing 'Home', 'Tools', and 'Add New Product'. Below the header, there is a table with columns 'Product Number' and 'Description'. The table contains 10 rows of product data. A modal form titled 'Add Product' is open in the center. The form has fields for 'Product Number', 'Description', 'Cost', 'UOM', and 'Category'. The 'Product Number' field contains 'P4999'. The 'Description', 'UOM', and 'Category' fields are empty. A red error notification box is displayed above the modal, containing the following messages: 'Description cannot be blank', 'Unit of Measure cannot be blank', and 'Category cannot be blank'. The modal form has 'Submit' and 'Cancel' buttons at the bottom.

Home Tools Add New Product

10 entries per page

Search:

Product Number Description

P0005 16GB US

P0013 AA Batta

P0008 Ballpoint

P0002 Bluetooth

P0006 Coffee M

P0009 Desk Lar

P0020 Electric E

P0004 Laptop B

P0016 LED Lig

P0019 Mechanic

Showing 1 to 10 of 20 entries

« < 1 2 > »

Add Product

Product Number: P4999

Description:

Cost: 22.99

UOM:

Category:

Submit Cancel

Description cannot be blank
Unit of Measure cannot be blank
Category cannot be blank

Category Actions

ronics

ronics

onery

ronics

en

e

en

ssories

e Improvement

onery

After we create the errors and click submit you will see a notification popup saying it was Successful and it's added to our HTML table.

Successfully added product: P4999

10 ▾ entries per page

Search:

Product Number	Description	Cost	UOM	Category	Actions
P0005	16GB USB Drive	9.99	EACH	Electronics	✎ 🗑
P4999	1TB External Drive	109.99	EACH	Electronic	✎ 🗑
P0013	AA Batteries (4 Pack)	4.25	PACK	Electronics	✎ 🗑
P0008	Ballpoint Pens (Pack of 10)	2.99	PACK	Stationery	✎ 🗑
P0002	Bluetooth Keyboard	29.95	EACH	Electronics	✎ 🗑
P0006	Coffee Mug	5.25	EACH	Kitchen	✎ 🗑
P0009	Desk Lamp	18.75	EACH	Office	✎ 🗑
P0020	Electric Kettle	24.5	EACH	Kitchen	✎ 🗑
P0004	Laptop Backpack	45	EACH	Accessories	✎ 🗑
P0016	LED Light Bulbs (6 Pack)	12.49	PACK	Home Improvement	✎ 🗑

Showing 1 to 10 of 21 entries

« ‹ 1 2 3 › »

What's Next

We've journeyed far together. Building tables, connecting Python and RPG, and crafting a working CRUD interface. While what we've built isn't production-ready, it has given us the fundamentals and shown what's possible. From here, the real adventure begins: exploring stronger frameworks like Flask, Gunicorn, FastAPI, and Nginx to create a more robust, scalable web solution for the IBM i.



And so, as our hero kneels before the castle, the princess places a sword upon his shoulder knighting him at last. This quest may pause here, but the path of discovery continues. The torch of knowledge has been passed, and the next chapter is yours to write.

In Closing

Every quest must reach a resting point, but the story never truly ends. Along the way we've faced dragons of complexity, crossed rivers of integration, and even sparred with sorcerers and dark knights of bad data. With each challenge, we've discovered that Python, RPG, and the IBM i can work together in powerful new ways.

Though these chapters close, the adventure continues. The tools and techniques we've explored here are just the beginning stepping stones. Whether your path takes you deeper into other Python frameworks or onward into your own unexplored realms, remember this: every line of code is a step forward, and every step brings you closer to mastery.

So raise your sword or your keyboard and keep going, brave adventurer. The quest is yours now.

I would like to mention if you haven't joined the Zulip chat group what are you waiting for. A lot of great gurus over there willing to help (<https://chat.ibmioss.org/>)

And if you enjoyed these articles drop me a note. I noticed the analytics went down the deeper we went. I'm not sure if it was the topic or the author, or LinkedIn algorithms.

Till next time.